AI/ML FRAMEWORKS

# TensorFlow

From tf.Tensor and GradientTape to Keras, tf.data pipelines, custom training loops, and deployment with SavedModel and TFLite.

tensorflow keras deep-learning tf.data gradienttape tflite ml-frameworks

Study Pack · Comprehensive reference & interview prep

## CONTENTS


1. 1. What TensorFlow Is and Why It Matters
2. 2. Setup and Getting Started
3. 3. Tensors: The Core Data Structure
4. 4. Automatic Differentiation with GradientTape
5. 5. tf.data: Input Pipelines That Keep the GPU Fed
6. 6. Building Models with the Keras API
7. 7. Training Models: compile / fit and Custom Loops
8. 8. tf.function and AutoGraph
9. 9. GPU Usage and Performance
10. 10. TensorBoard: Visualizing Training
11. 11. Saving and Loading Models
12. 12. Deployment: SavedModel, TF Serving, and TFLite
13. 13. Worked Mini-Example: MNIST End to End
14. 14. Cheat Sheet and Comparison
15. 15. Best Practices and Common Pitfalls
16. 16. Interview Questions & Answers
17. 17. Further Learning


TensorFlow is Google's end-to-end platform for building, training, and deploying machine learning models — from a one-line Keras classifier on your laptop to a multi-GPU training job feeding a model that runs on a phone, a browser, or a production server. This guide takes you from the humble `tf.Tensor` all the way to a SavedModel and a quantized `.tflite` binary, with the mental models and code you need for both real work and interviews.

# 1. What TensorFlow Is and Why It Matters

At its core TensorFlow is a library for **numerical computation over multidimensional arrays (tensors)**, with two super-powers layered on top: **automatic differentiation** (compute gradients of any computation) and **hardware acceleration** (run the same code on CPU, GPU, or TPU). Keras — the high-level API bundled inside TensorFlow — gives you a clean, batteries-included way to assemble and train neural networks on top of that engine.

### EAGER BY DEFAULT

Since TF 2.x, operations run immediately like NumPy. No sessions, no placeholders — you debug with `print()`.

### GRAPHS WHEN YOU WANT THEM

`@tf.function` traces your Python into a fast, portable graph for speed and deployment — opt-in, not mandatory.

### DEPLOY EVERYWHERE

SavedModel for servers, TFLite for mobile/embedded, TensorFlow.js for browsers, TF Serving for scalable APIs.

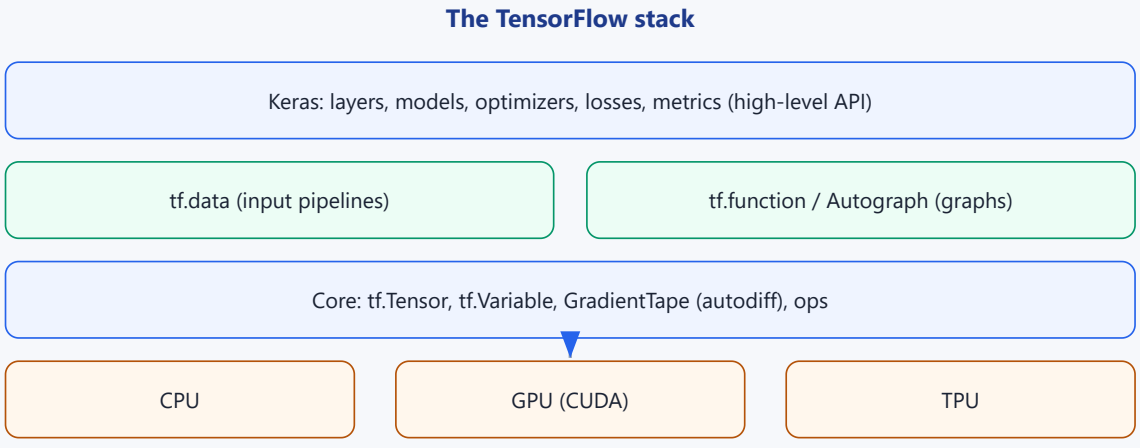



*Fig 1. Layers of the TensorFlow stack: Keras sits on tf.data and tf.function, which sit on the core tensor engine, which maps to hardware.*

**TF 1 vs TF 2**

If you see `tf.Session()`, `tf.placeholder`, or `feed_dict` in a tutorial, it is TensorFlow 1.x. TF 2.x (current as of 2026, with the 2.x series having matured through Keras 3 multi-backend support) is eager-first and Keras-centric. This guide is pure TF 2.x.

## 2. Setup and Getting Started

TensorFlow ships as a pip package. The same `tensorflow` wheel includes CPU and GPU support on Linux; verify your CUDA/cuDNN drivers for GPU.

BASH

```bash
# Create an isolated environment (recommended)
python -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate

# Install TensorFlow (CPU + GPU on Linux via the same wheel)
pip install --upgrade tensorflow

# Verify the install and list visible GPUs
python -c "import tensorflow as tf; print(tf.__version__); print(tf.config.list_physic
```

PYTHON

```python
import tensorflow as tf

print(tf.__version__)                                  # e.g. 2.18.0
print("GPUs:", tf.config.list_physical_devices("GPU"))
print("Eager execution:", tf.executing_eagerly())   # True in TF 2.x
```

✦ **Reproducibility**

Seed everything up front: `tf.keras.utils.set_random_seed(42)` seeds Python, NumPy, and TensorFlow in one call. For bit-exact GPU runs add `tf.config.experimental.enable_op_determinism()` (slower, but deterministic).

## 3. Tensors: The Core Data Structure

A **tensor** is an immutable n-dimensional array with a uniform `dtype` and a `shape`. Rank is the number of dimensions: a scalar is rank 0, a vector rank 1, a matrix rank 2, and so on.

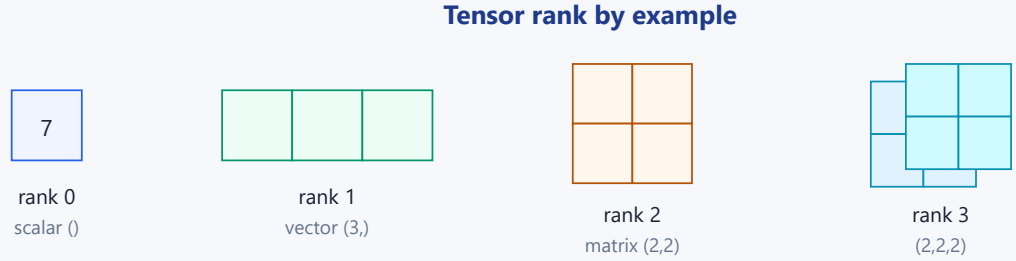



*Fig 2. Rank = number of axes. An image batch is typically rank 4: (batch, height, width, channels).*

PYTHON

```python
import tensorflow as tf

# Create tensors
scalar = tf.constant(7)                          # shape (), int32
vec    = tf.constant([1.0, 2.0, 3.0])            # shape (3,), float32
mat    = tf.constant([[1, 2], [3, 4]])           # shape (2, 2)

print(mat.shape, mat.dtype, tf.rank(mat).numpy())  # (2, 2) int32 2

# Element-wise ops and broadcasting (like NumPy)
a = tf.constant([[1.0, 2.0], [3.0, 4.0]])
b = tf.constant([10.0, 20.0])                    # broadcast across rows
print(a + b)                                     # [[11, 22], [13, 24]]

# Matrix multiply, reduction, reshape
print(tf.matmul(a, a))                           # (2,2) @ (2,2)
print(tf.reduce_mean(a, axis=0))                 # column means → [2., 3.]
print(tf.reshape(a, (1, 4)))                     # (1, 4)

# Interop with NumPy is zero-copy on CPU
import numpy as np
np_arr = a.numpy()                               # tensor → ndarray
back   = tf.convert_to_tensor(np_arr)            # ndarray → tensor
```

## Variables: mutable, trainable state

Tensors are immutable. A `tf.Variable` wraps a tensor you can mutate in place — this is how model weights live and get updated.

PYTHON

```python
w = tf.Variable([[1.0, 2.0]], name="weights")   # trainable by default
w.assign([[0.5, 0.5]])          # replace value
w.assign_add([[0.1, 0.1]])      # in-place add → [[0.6, 0.6]]
print(w.trainable)              # True — tracked by GradientTape
```

> ⚠ **dtype matters**
>
> Mixing dtypes raises errors: `tf.constant(1) + tf.constant(1.0)` fails (int32 vs float32). Cast explicitly with `tf.cast(x, tf.float32)`. Most model math uses `float32`; mixed-precision training uses `float16` / `bfloat16` for speed.

## 4. Automatic Differentiation with GradientTape

`tf.GradientTape` records operations onto a "tape" so TensorFlow can replay them in reverse to compute gradients. This is the engine under every optimizer.

PYTHON

```python
# Minimize f(x) = x^2 — manual gradient descent
x = tf.Variable(5.0)
lr = 0.1

for step in range(20):
    with tf.GradientTape() as tape:
        y = x ** 2                  # forward pass is recorded
    dy_dx = tape.gradient(y, x)     # dy/dx = 2x
    x.assign_sub(lr * dy_dx)        # x -= lr * grad

print(round(x.numpy(), 4))          # approaches 0.0
```

PYTHON

```python
# Gradients of a loss w.r.t. many parameters at once
w = tf.Variable(tf.random.normal((3, 1)))
b = tf.Variable(tf.zeros((1,)))
X = tf.constant([[1., 2., 3.], [4., 5., 6.]])
y = tf.constant([[1.], [0.]])

with tf.GradientTape() as tape:
    pred = tf.matmul(X, w) + b
    loss = tf.reduce_mean((pred - y) ** 2)      # MSE

grads = tape.gradient(loss, [w, b])             # list of gradients
print(grads[0].shape, grads[1].shape)           # (3,1) (1,)
```

✦ **Persistent tapes**

By default a tape is consumed after one `gradient()` call. Use `tf.GradientTape(persistent=True)` to take multiple gradients from the same tape (e.g. higher-order derivatives), and `del tape` to free it afterward. Call `tape.watch(t)` to track a constant tensor that is not a `Variable`.

## 5. tf.data: Input Pipelines That Keep the GPU Fed

Data loading is the most common bottleneck. `tf.data.Dataset` builds a lazy, composable pipeline that reads, transforms, shuffles, batches, and prefetches data in parallel on the CPU while the GPU trains on the previous batch.




*Fig 3. A canonical tf.data pipeline. Prefetch overlaps CPU preprocessing with GPU compute so neither waits.*

PYTHON

```python
AUTOTUNE = tf.data.AUTOTUNE

ds = tf.data.Dataset.from_tensor_slices((features, labels))

ds = (ds
      .shuffle(buffer_size=10_000)              # shuffle before batching
      .map(preprocess, num_parallel_calls=AUTOTUNE)
      .batch(32)
      .prefetch(AUTOTUNE))                       # overlap CPU + GPU work

for batch_x, batch_y in ds.take(1):
    print(batch_x.shape, batch_y.shape)

# Reading real files efficiently
files = tf.data.Dataset.list_files("data/*.tfrecord")
raw   = tf.data.TFRecordDataset(files, num_parallel_reads=AUTOTUNE)

# Cache after expensive deterministic steps to skip recomputation each epoch
ds = ds.cache()        # in memory, or ds.cache("/tmp/cache") on disk
```

⚠ **Order of operations**

Call `.shuffle()` *before* `.batch()` (otherwise you only shuffle batch order), put `.cache()` after deterministic maps but before random augmentation, and always finish with `.prefetch(AUTOTUNE)`. A buffer that is too small defeats shuffling; size it to a large fraction of the dataset when memory allows.

# 6. Building Models with the Keras API

Keras offers three model-building styles, trading simplicity for flexibility.

### SEQUENTIAL

A linear stack of layers. Simplest; one input, one output, no branching.

### FUNCTIONAL

A graph of layers. Supports multiple inputs/outputs, shared layers, residual connections.

### SUBCLASSING

Write a `Model` subclass with a custom `call()`. Maximum flexibility for research and dynamic control flow.

PYTHON

```python
from tensorflow import keras
from tensorflow.keras import layers

# --- Sequential API ---
model = keras.Sequential([
    keras.Input(shape=(28, 28)),
    layers.Flatten(),
    layers.Dense(128, activation="relu"),
    layers.Dropout(0.2),
    layers.Dense(10, activation="softmax"),
])
model.summary()                      # prints layers, output shapes, param counts
```

PYTHON

```python
# --- Functional API: two-branch model with a skip connection ---
inputs = keras.Input(shape=(32,))
x = layers.Dense(64, activation="relu")(inputs)
y = layers.Dense(64, activation="relu")(x)
z = layers.Add()([x, y])             # residual: combine two tensors
outputs = layers.Dense(1)(z)
model = keras.Model(inputs, outputs, name="residual_mlp")
```

PYTHON

```python
# --- Subclassing API: full control over the forward pass ---
class MLP(keras.Model):
    def __init__(self, units=64, classes=10):
        super().__init__()
        self.d1 = layers.Dense(units, activation="relu")
        self.drop = layers.Dropout(0.3)
        self.out = layers.Dense(classes, activation="softmax")

    def call(self, x, training=False):
        x = self.d1(x)
        x = self.drop(x, training=training)   # dropout only during training
        return self.out(x)

model = MLP()
```

# 7. Training Models: compile / fit and Custom Loops

The high-level path is `compile()` + `fit()`. You specify an optimizer, a loss, and metrics; Keras runs the training loop, validation, and callbacks for you.

PYTHON

```
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss="sparse_categorical_crossentropy",   # integer labels
    metrics=["accuracy"],
)

callbacks = [
    keras.callbacks.EarlyStopping(patience=3, restore_best_weights=True),
    keras.callbacks.ModelCheckpoint("best.keras", save_best_only=True),
    keras.callbacks.TensorBoard(log_dir="logs/fit"),
]

history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=20,
    callbacks=callbacks,
)

test_loss, test_acc = model.evaluate(test_ds)
preds = model.predict(new_data)              # forward pass, no gradients
```

**Pick the right loss**

Use `sparse_categorical_crossentropy` for integer class labels, `categorical_crossentropy` for one-hot labels, `binary_crossentropy` for 0/1 or multi-label, and `mse` / `mae` for regression. Mismatching label format and loss is a classic silent bug.

## Custom training loops

When you need something `fit()` cannot express (GANs, custom gradient manipulation, reinforcement learning), write the loop yourself with `GradientTape`.

PYTHON

```python
optimizer = keras.optimizers.Adam(1e-3)
loss_fn   = keras.losses.SparseCategoricalCrossentropy()
train_acc = keras.metrics.SparseCategoricalAccuracy()

@tf.function                              # compile the step into a graph
def train_step(x, y):
    with tf.GradientTape() as tape:
        logits = model(x, training=True)
        loss = loss_fn(y, logits)
    grads = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(grads, model.trainable_variables))
    train_acc.update_state(y, logits)
    return loss

for epoch in range(EPOCHS):
    train_acc.reset_state()
    for x, y in train_ds:
        loss = train_step(x, y)
    print(f"epoch {epoch}: loss={loss:.4f} acc={train_acc.result():.4f}")
```

## 8. tf.function and AutoGraph

`@tf.function` traces a Python function once, converting it into a static **computation graph** that runs without Python overhead and can be serialized for deployment. AutoGraph rewrites Python control flow ( `if` , `for` , `while` ) into graph ops automatically.

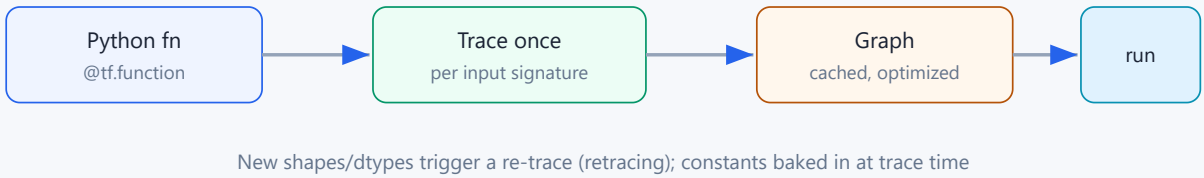



*Fig 4. tf.function traces the Python once per input signature, caches the graph, then runs the graph on later calls.*

PYTHON

```python
@tf.function
def f(x):
    if tf.reduce_sum(x) > 0:        # AutoGraph converts this to tf.cond
        return x * 2
    return x - 1

print(f(tf.constant([1.0, 2.0])))   # runs as a graph

# Inspect the generated graph code
print(tf.autograph.to_code(f.python_function))
```

**Tracing pitfalls**

Python side effects ( `print` , list `.append` , incrementing a Python counter) run only *during tracing*, not on every call. Pass variable-length inputs as tensors, not Python lists, to avoid a re-trace per shape. Use `tf.print` (not `print` ) to log at run time, and `input_signature` to pin shapes and prevent retracing.

## 9. GPU Usage and Performance

TensorFlow places ops on a GPU automatically when one is visible. You control memory and device placement explicitly for multi-GPU and shared machines.

PYTHON

```python
# Grow GPU memory on demand instead of grabbing it all at startup
gpus = tf.config.list_physical_devices("GPU")
for gpu in gpus:
    tf.config.experimental.set_memory_growth(gpu, True)

# Pin a computation to a specific device
with tf.device("/GPU:0"):
    a = tf.random.normal((1000, 1000))
    b = tf.matmul(a, a)

# Mixed precision: float16 math, float32 master weights → big speedup on modern GPUs
keras.mixed_precision.set_global_policy("mixed_float16")

# Multi-GPU data parallelism with one context manager
strategy = tf.distribute.MirroredStrategy()
with strategy.scope():
    model = build_model()           # variables mirrored across GPUs
    model.compile(optimizer="adam", loss="mse")
model.fit(train_ds, epochs=10)      # batches split across GPUs
```

**Keep the GPU busy**

If GPU utilization is low, the bottleneck is usually the input pipeline. Add `.prefetch(AUTOTUNE)`, parallelize `.map()`, `.cache()`, and increase batch size. Use the TensorBoard Profiler to see the exact gap between data and compute.

## 10. TensorBoard: Visualizing Training

TensorBoard reads event logs written during training and renders scalars (loss/accuracy curves), histograms, the model graph, images, and profiling traces.

PYTHON

```python
# 1. Add the callback (writes to log_dir)
tb = keras.callbacks.TensorBoard(log_dir="logs/fit", histogram_freq=1)
model.fit(train_ds, validation_data=val_ds, epochs=10, callbacks=[tb])

# 2. In a custom loop, write scalars yourself
writer = tf.summary.create_file_writer("logs/custom")
with writer.as_default():
    tf.summary.scalar("loss", loss, step=epoch)
```

BASH

```bash
# 3. Launch the dashboard, then open http://localhost:6006
tensorboard --logdir logs/fit
```

## 11. Saving and Loading Models

There are two things you may want to persist: the **full model** (architecture + weights + optimizer state) or just the **weights**. TF 2.x offers the native `.keras` format, the language-neutral **SavedModel** directory, and raw weight checkpoints.

| Format | Call | Contains | Use for |
|---|---|---|---|
| `.keras` (recommended) | `model.save("m.keras")` | Arch + weights + optimizer | Resume training, share Keras models |
| SavedModel (dir) | `model.export("m/")` | Graph + weights, no Python | TF Serving, TFLite, cross-language |
| Weights only | `model.save_weights("w.h5")` | Weights only | Transfer learning, custom code |
| Checkpoints | `tf.train.Checkpoint` | Any trackable objects | Custom loops, fault tolerance |

PYTHON

```python
# Native Keras format (single file) - best for Keras-to-Keras
model.save("mymodel.keras")
restored = keras.models.load_model("mymodel.keras")

# SavedModel - portable graph for serving / TFLite (no Python needed)
model.export("saved_model_dir")
loaded = tf.saved_model.load("saved_model_dir")
infer = loaded.signatures["serving_default"]

# Weights only
model.save_weights("weights.weights.h5")
model.load_weights("weights.weights.h5")   # architecture must already exist
```

> **Custom layers**
> If your model uses custom layers or losses, pass them when loading: `load_model("m.keras", custom_objects={"MyLayer": MyLayer})`, or decorate the class with `@keras.saving.register_keras_serializable()` so Keras finds it automatically.

# 12. Deployment: SavedModel, TF Serving, and TFLite

Training is half the job. Deployment turns a model into something that answers requests on a server, a phone, or an edge device.

**Trained model** -> **SavedModel** -> **TF Serving / TFLite / TF.js** -> **Predictions**

## TF Serving (scalable server-side)

BASH

```bash
# Serve a SavedModel over gRPC + REST with the official Docker image
docker run -p 8501:8501 \
  --mount type=bind,source=$(pwd)/saved_model_dir,target=/models/mymodel \
  -e MODEL_NAME=mymodel tensorflow/serving

# Query the REST endpoint
curl -d '{"instances": [[1.0, 2.0, 3.0]]}' \
  -X POST http://localhost:8501/v1/models/mymodel:predict
```

## TFLite (mobile and embedded)

PYTHON

```python
# Convert a SavedModel to a compact, quantized .tflite binary
converter = tf.lite.TFLiteConverter.from_saved_model("saved_model_dir")
converter.optimizations = [tf.lite.Optimize.DEFAULT]   # dynamic-range quantization
tflite_model = converter.convert()
with open("model.tflite", "wb") as f:
    f.write(tflite_model)

# Run inference with the TFLite interpreter
interpreter = tf.lite.Interpreter(model_path="model.tflite")
interpreter.allocate_tensors()
inp = interpreter.get_input_details()[0]
out = interpreter.get_output_details()[0]
interpreter.set_tensor(inp["index"], sample_input)
interpreter.invoke()
print(interpreter.get_tensor(out["index"]))
```

✦ **Quantization trade-off**

Dynamic-range quantization shrinks a model ~4x and speeds up CPU inference with minimal accuracy loss. Full integer quantization (int8) is smaller and faster still but needs a representative dataset for calibration — essential for microcontrollers.

## 13. Worked Mini-Example: MNIST End to End

A complete, runnable classifier tying together data loading, a Keras model, training, evaluation, and export.

PYTHON

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

tf.keras.utils.set_random_seed(42)

# 1. Load and normalize data
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
x_train, x_test = x_train / 255.0, x_test / 255.0

# 2. Build a tf.data pipeline
AUTOTUNE = tf.data.AUTOTUNE
train_ds = (tf.data.Dataset.from_tensor_slices((x_train, y_train))
            .shuffle(10_000).batch(128).prefetch(AUTOTUNE))
test_ds  = (tf.data.Dataset.from_tensor_slices((x_test, y_test))
            .batch(128).prefetch(AUTOTUNE))

# 3. Define the model
model = keras.Sequential([
    keras.Input((28, 28)),
    layers.Flatten(),
    layers.Dense(128, activation="relu"),
    layers.Dropout(0.2),
    layers.Dense(10, activation="softmax"),
])

# 4. Compile and train
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
model.fit(train_ds, validation_data=test_ds, epochs=5)

# 5. Evaluate and export
loss, acc = model.evaluate(test_ds)
print(f"test accuracy: {acc:.4f}")           # ~0.98
model.save("mnist.keras")
```

# 14. Cheat Sheet and Comparison

| Task | Code |
|---|---|
| Create constant / variable | `tf.constant(x)` / `tf.Variable(x)` |
| Shape, rank, dtype | `t.shape`, `tf.rank(t)`, `t.dtype` |
| Cast dtype | `tf.cast(t, tf.float32)` |
| Reshape / expand / squeeze | `tf.reshape`, `tf.expand_dims`, `tf.squeeze` |
| Matrix multiply | `tf.matmul(a, b)` or `a @ b` |
| Reduce | `tf.reduce_sum/mean/max(t, axis=...)` |
| Gradient | `tape.gradient(loss, vars)` |
| Dataset from arrays | `tf.data.Dataset.from_tensor_slices(...)` |
| Compile + train | `model.compile(...)` ; `model.fit(...)` |
| Save / load | `model.save("m.keras")` / `load_model(...)` |
| List GPUs | `tf.config.list_physical_devices("GPU")` |

### TENSORFLOW VS PYTORCH AT A GLANCE

| Aspect | TensorFlow 2.x | PyTorch |
|---|---|---|
| Execution | Eager + `@tf.function` graphs | Eager + `torch.compile` |
| High-level API | Keras (built in) | `torch.nn` + Lightning (3rd party) |
| Autodiff | `GradientTape` | `loss.backward()` / autograd |
| Mobile/edge | TFLite (mature) | ExecuTorch / PyTorch Mobile |
| Serving | TF Serving | TorchServe |
| Typical strength | Production, mobile, TPU | Research, flexibility |

# 15. Best Practices and Common Pitfalls

### DO

Use `tf.data` with `prefetch(AUTOTUNE)`; use `@tf.function` for training steps; normalize inputs; use callbacks for checkpointing and early stopping; seed with `set_random_seed`.

### AVOID

Python loops over tensors; growing Python lists inside `tf.function`; mixing dtypes; shuffling after batching; grabbing all GPU memory on shared machines.

✓ **Prefer vectorized ops**

Replace per-element Python loops with tensor ops ( `tf.reduce_sum` , broadcasting, `tf.where` ). Vectorized code runs on the GPU in parallel; Python loops serialize on the CPU and are often 100x slower.

⚠ **Validation data leakage**

Fit normalization statistics (mean/std, min/max) on the *training* split only, then apply them to validation and test. Use a `keras.layers.Normalization` layer with `adapt()` on train data so the stats travel with the saved model.

✕ **Silent shape bugs**

Broadcasting can hide bugs: subtracting a `(32,)` tensor from a `(32,1)` tensor produces a `(32,32)` result instead of an error. Assert shapes with `tf.debugging.assert_shapes` and watch `model.summary()` output shapes.

# 16. Interview Questions & Answers

**What changed between TensorFlow 1.x and 2.x?**

TF 2.x makes **eager execution** the default (ops run immediately, debug with Python tools), removes `Session` / `placeholder` / `feed_dict`, promotes Keras to the official high-level API, and replaces manual graph construction with opt-in `@tf.function` tracing. The result is a far more Pythonic, debuggable experience with graph performance available on demand.

**Difference between a tf.Tensor and a tf.Variable?**

A `Tensor` is immutable — ops produce new tensors. A `Variable` is mutable state you update in place with `assign` / `assign_add`. Model weights are `Variable`s because the optimizer must update them each step, and the `GradientTape` tracks trainable variables automatically.

### How does GradientTape compute gradients?

It records every differentiable op executed inside its context onto a tape. Calling `tape.gradient(y, x)` replays the tape in reverse (reverse-mode automatic differentiation / backpropagation), applying the chain rule to accumulate `dy/dx`. By default it watches all trainable `Variable`s; use `tape.watch()` for constants and `persistent=True` for multiple gradient calls.

### What does @tf.function actually do, and when should you use it?

It traces the Python function into a static TensorFlow graph (via AutoGraph for control flow), caches that graph per input signature, and runs the graph on subsequent calls — eliminating Python overhead and enabling serialization and cross-device optimization. Use it on hot paths like the training step. Beware: Python side effects run only during tracing, and new input shapes trigger expensive re-tracing.

### Why use tf.data instead of feeding NumPy arrays directly?

`tf.data` builds a lazy, parallel pipeline that reads, transforms, shuffles, batches, and prefetches on the CPU while the GPU trains the previous batch. It scales beyond memory (streaming from TFRecords), parallelizes mapping with `AUTOTUNE`, and overlaps I/O with compute via `prefetch` — keeping the GPU saturated instead of idle.

### Why does prefetch go last and shuffle before batch?

`prefetch` decouples producer (CPU) from consumer (GPU), so it must sit at the end to buffer finished batches. `shuffle` must come before `batch` so individual examples are randomized; shuffling after batching only reorders whole batches, leaving each batch's contents fixed and hurting training.

### When would you write a custom training loop instead of model.fit?

When the standard loop cannot express your needs: GANs (two optimizers, alternating updates), reinforcement learning, custom gradient clipping or accumulation, multiple losses with per-step weighting, or research experiments. You get full control via `GradientTape` + `optimizer.apply_gradients`, at the cost of writing the loop, metrics, and checkpointing yourself.

### Sequential vs Functional vs Subclassing Keras APIs?

**Sequential**: a simple linear stack, one input/output. **Functional**: a graph of layers supporting multiple inputs/outputs, shared layers, and skip connections while staticly validating shapes. **Subclassing**: a `Model` with a custom `call()` for dynamic/imperative logic — most flexible but no static graph checking until the first call.

**What is the difference between the .keras format and a SavedModel?**

The `.keras` file is a Keras-native archive (architecture + weights + optimizer state) ideal for pausing/resuming and sharing Keras models. A **SavedModel** is a language-neutral directory containing the serialized computation graph and weights with no Python dependency — the format TF Serving, TFLite, and other languages consume for deployment.

**How do you convert a model for mobile, and what is quantization?**

Use `tf.lite.TFLiteConverter.from_saved_model( ... )` to produce a compact `.tflite` binary run by the lightweight interpreter. Quantization reduces weight/activation precision (e.g. float32 to int8), shrinking the model ~4x and speeding up CPU/edge inference. Dynamic-range quantization is nearly free; full-integer quantization needs a representative dataset for calibration.

**How do you train across multiple GPUs?**

Wrap model creation and compilation in a `tf.distribute.MirroredStrategy().scope()`. The strategy mirrors variables on each GPU, splits each global batch across devices (data parallelism), computes gradients locally, and all-reduces them to keep replicas in sync. For multi-machine training use `MultiWorkerMirroredStrategy`; for TPUs, `TPUStrategy`.

**Your GPU utilization is low during training — how do you diagnose it?**

Almost always the input pipeline is starving the GPU. Add `.prefetch(AUTOTUNE)`, parallelize `.map(num_parallel_calls=AUTOTUNE)`, `.cache()` deterministic work, and increase batch size. Confirm with the TensorBoard Profiler, which shows the timeline gap between input and compute. Also check for expensive Python ops inside `tf.function` or frequent retracing.

# 17. Further Learning

- **Official guides** — tensorflow.org/guide: the `tf.data`, `tf.function`, and distributed training guides are excellent and current.
- **Keras 3** — the multi-backend Keras (TensorFlow, JAX, PyTorch); learn the stable API that outlives any one backend.
- **TensorBoard Profiler** — master it to find input bottlenecks and GPU idle time.
- **Model optimization toolkit** — pruning, quantization-aware training, and clustering for smaller, faster models.
- **TFX** — TensorFlow Extended for production ML pipelines (data validation, training, serving) when you move toward MLOps.
- **Build a project** — take a dataset from `tf.data` through training, TensorBoard, and a TFLite export to cement the full lifecycle.